

Memoria de la Práctica – Procesadores de Lenguajes (Curso 2024/2025)

Objetivo de la práctica

El objetivo de esta práctica ha sido ampliar la funcionalidad del analizador calc7 para construir una nueva versión, calc8, capaz de generar un Árbol de Sintaxis Abstracta (AST) binario a partir de expresiones aritméticas. El AST se debe imprimir en notación prefija estilo LISP. Además, se debía permitir la evaluación de expresiones con asignaciones encadenadas y variables.

Cambios implementados respecto a calc7

1. **Definición de una estructura para el AST:** Se definió la estructura ASTnode para representar los nodos del árbol:

```
struct ASTnode {  
    char *op;  
  
    int type; // 0: hoja, 1: unario, 2: binario  
  
    t_node *left;  
  
    t_node *right;  
  
};
```

2. **Creación y liberación del AST:**

- Se implementó la función createASTNode para crear nodos.
- Se implementó freeAST para liberar la memoria recursivamente.

3. **Impresión en notación prefija:**

- La función printAST2Prefix permite recorrer el AST e imprimirlo estilo LISP.

4. **Extensión del tipo semántico YYSTYPE:**

- Se definió una estructura t_attr con un puntero node para manipular el AST dentro de las acciones semánticas:

```
typedef struct {  
    int value;  
  
    int index;  
  
    char *code;  
  
    t_node *node;  
  
} t_attr;  
  
#define YYSTYPE t_attr
```

5. **Redefinió el axioma para permitir múltiples líneas:**

- Se modificó la regla axiom para que pueda aceptar una o más expresiones separadas por saltos de línea:

axiom:

```
/* vacío */  
| axiom expression '\n' {  
    printAST2Prefix($2.node);  
    printf("\n");  
    freeAST($2.node);  
}  
;
```

6. Asignaciones encadenadas:

- Se incluyó en la regla expression la producción:

```
| VARIABLE '=' expression {  
    char *name = char_to_string($1.index + 'A');  
    $$$.node = createASTNode("setq", 2, createASTNode(name, 0, NULL, NULL),  
    $3.node);  
}
```

- Se declaró %right '=' para permitir asociatividad a derecha.

7. Corrección del analizador léxico (yylex):

- Se añadió <ctype.h> para usar isdigit e isalpha.
- Se implementó el reconocimiento de:
 - Números (NUMBER)
 - Variables (VARIABLE) mayúsculas o minúsculas
 - Operadores, paréntesis y saltos de línea

```
if (isdigit(c)) {  
    ungetc(c, stdin);  
    scanf("%d", &yylval.value);  
    return NUMBER;  
}
```

```
if (isalpha(c)) {  
    yylval.index = (c & ~32) - 'A';
```

```
    return VARIABLE;  
}
```

Casos de prueba utilizados

Para verificar el funcionamiento del analizador, se han probado las siguientes entradas:

42

A

1 + 2

3 * 4

2 + 3 * 4

(2 + 3) * 4

1 + 2 + 3

-5

-A + 4

A = 10

B = 2 * (3 + 4)

A = B + -3 * (C - 1)

a = b = c = 2 + 3

Todas ellas han sido procesadas correctamente, generando la salida en forma de AST prefijo.
Por ejemplo:

Entrada:

a = b = c = 2 + 3

Salida:

(setq a (setq b (setq c (+ 2 3))))

También, se han reflejado las salidas esperadas del archivo de prueba en un `esperado.txt` que reflejan todas las soluciones devueltas de la implementación.

Como hablamos en clase, mi ordenador tenía problemas para correr el archivo de pruebas pero me comentaste que te lo entregase así porque funcionaba correctamente cuando se probaban individualmente.

Conclusión

La práctica ha permitido aplicar de forma práctica los conocimientos de construcción de analizadores sintácticos con Bison y el diseño de estructuras árboles. Se ha trabajado la

Alberto Menchén y Jorge Mejías

precedencia y asociatividad de operadores, el análisis de expresiones complejas y la manipulación de memoria dinámica para representar árboles sintácticos.